

CS339: Abstractions and Paradigms for Programming

Au Revoir (until we meet again)

Manas Thakur
CSE, IIT Bombay



Autumn 2025

The **amb** operator from SICP

- (amb a b c) chooses any of the values a, b or c
 - (amb) returns false (no choices)*
- Introduces *ambiguity* in our programs:

```
(define (an-integer-between low high)
  (if (> low high)
      (amb)
      (amb low (an-integer-between (+ low 1) high))))

(an-integer-between 1 3)
```

```
(let ((x (an-integer-between 1 10))
      (y (an-integer-between 1 10)))
  (require (= (+ x y) 10))
  (list x y))
```

```
(define (require p)
  (if (not p) (amb)))
```

*The amb obtained using #lang sicp in DrRacket is different.



Ambiguity -> Non-determinism -> Concurrency

- Non-deterministic paradigm invented by John McCarthy in 1961
- What if we could explore the search space in parallel?

➤ *Difference between parallel and concurrent?*

```
(define x 0)

(parallel-execute
  (lambda () (set! x (+ x 1)))
  (lambda () (set! x (* x 2))))
```

- May lead to a **data race**:
 - If two accesses to the same memory location *may-happen-in-parallel* and at least one of them is a write (variable level, field level, cache level, page level, ...)
- How do we mitigate data races?
- Which ones should we try to mitigate?



The Concurrency Paradigm



Concurrent Programs

Initially, $A = B = 0$.

Thread 1

(1) $A = 1$
(2) print B

Thread 2

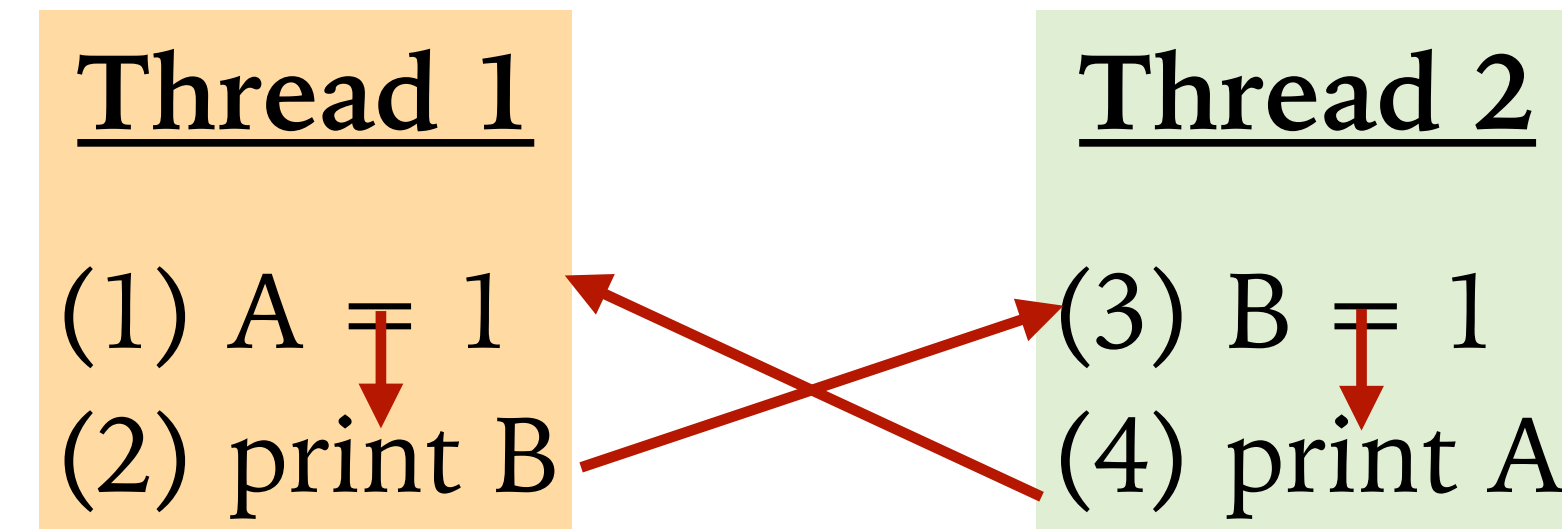
(3) $B = 1$
(4) print A

- What all can be printed?
 - (1) $\rightarrow$ (2) $\rightarrow$ (3) $\rightarrow$ (4): **01**
 - (3) $\rightarrow$ (4) $\rightarrow$ (1) $\rightarrow$ (2): **01**
 - (1) $\rightarrow$ (3) $\rightarrow$ (2) $\rightarrow$ (4): **11**
 - (1) $\rightarrow$ (3) $\rightarrow$ (4) $\rightarrow$ (2): **11**
 - A few others with the same effect.



Things that shouldn't happen

- It should not be possible to print **00**.



- In order to print **00**:
 - For line (2) to print 0, (2) should *happen before* (3) writes 1 to B.
 - For line (4) to print 0, (4) should happen before (1) writes 1 to A.
 - Each thread's events should be in order: (1) before (2) and (3) before (4).
 - This implies (1) should happen before (1)!
 - A contradiction — means **00 cannot be printed** by this multithreaded program.



Sequential Consistency

1. All operations executed in some sequential order.
 - As if they were manipulating a *single shared memory*.
2. Each thread's operations happen in program order.

Initially, $A = B = 0$.

Thread 1

$A = 1$
 $r0 = B$

Thread 2

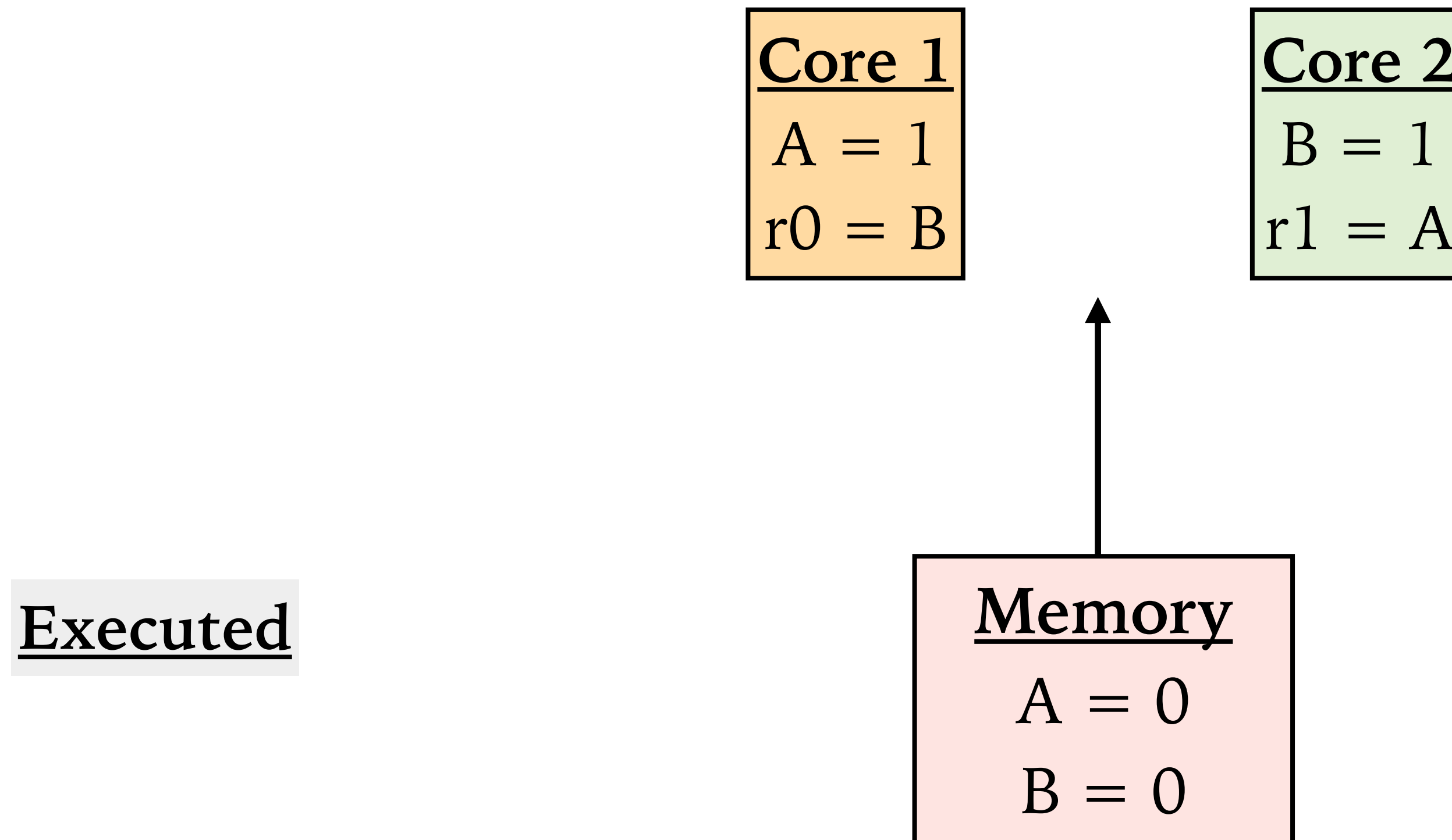
$B = 1$
 $r1 = A$

- Not allowed: $r0 = 0$ and $r1 = 0$.



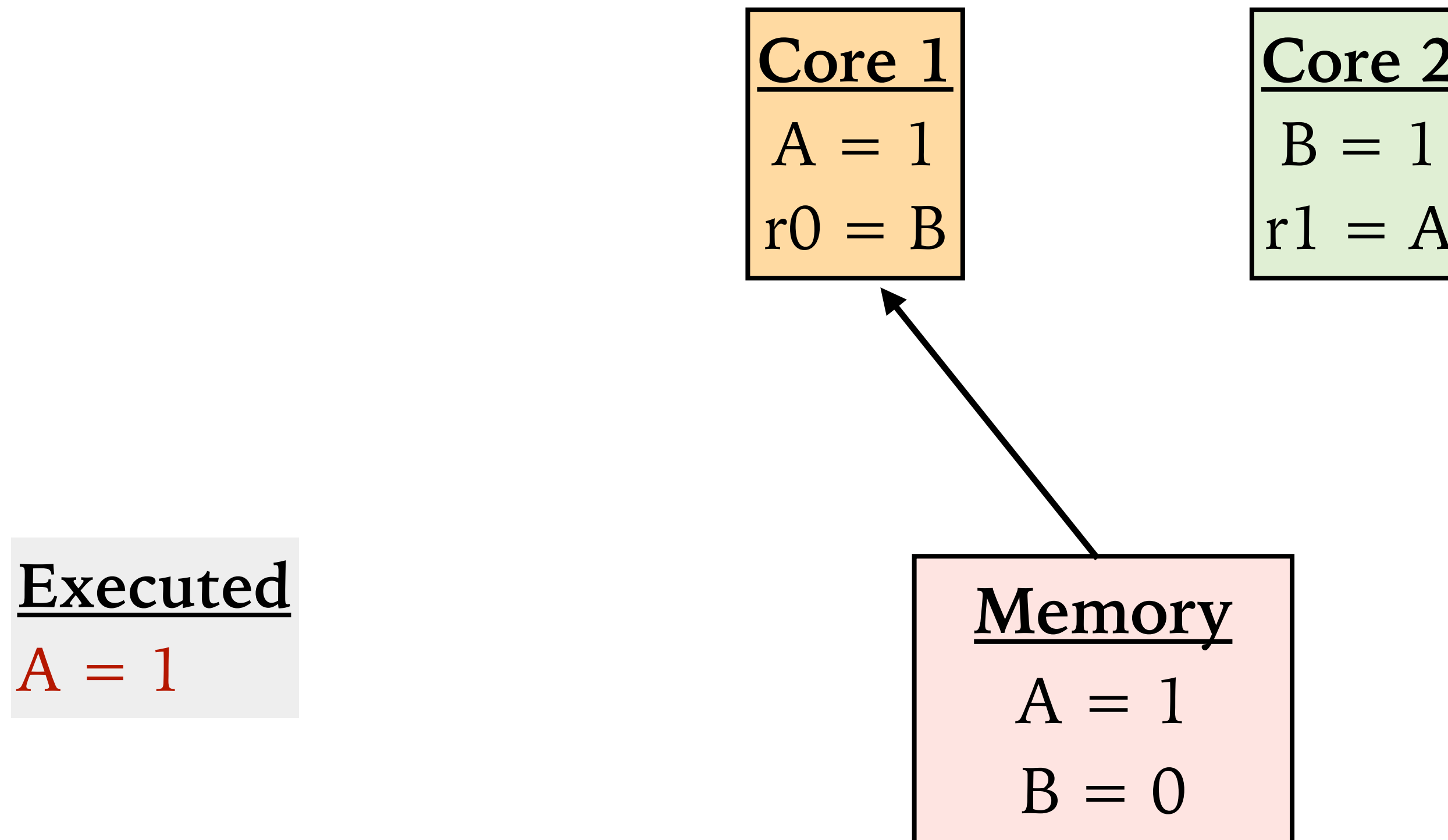
Sequential Consistency

- Can be seen as a “switch” running one instruction at a time from different threads.



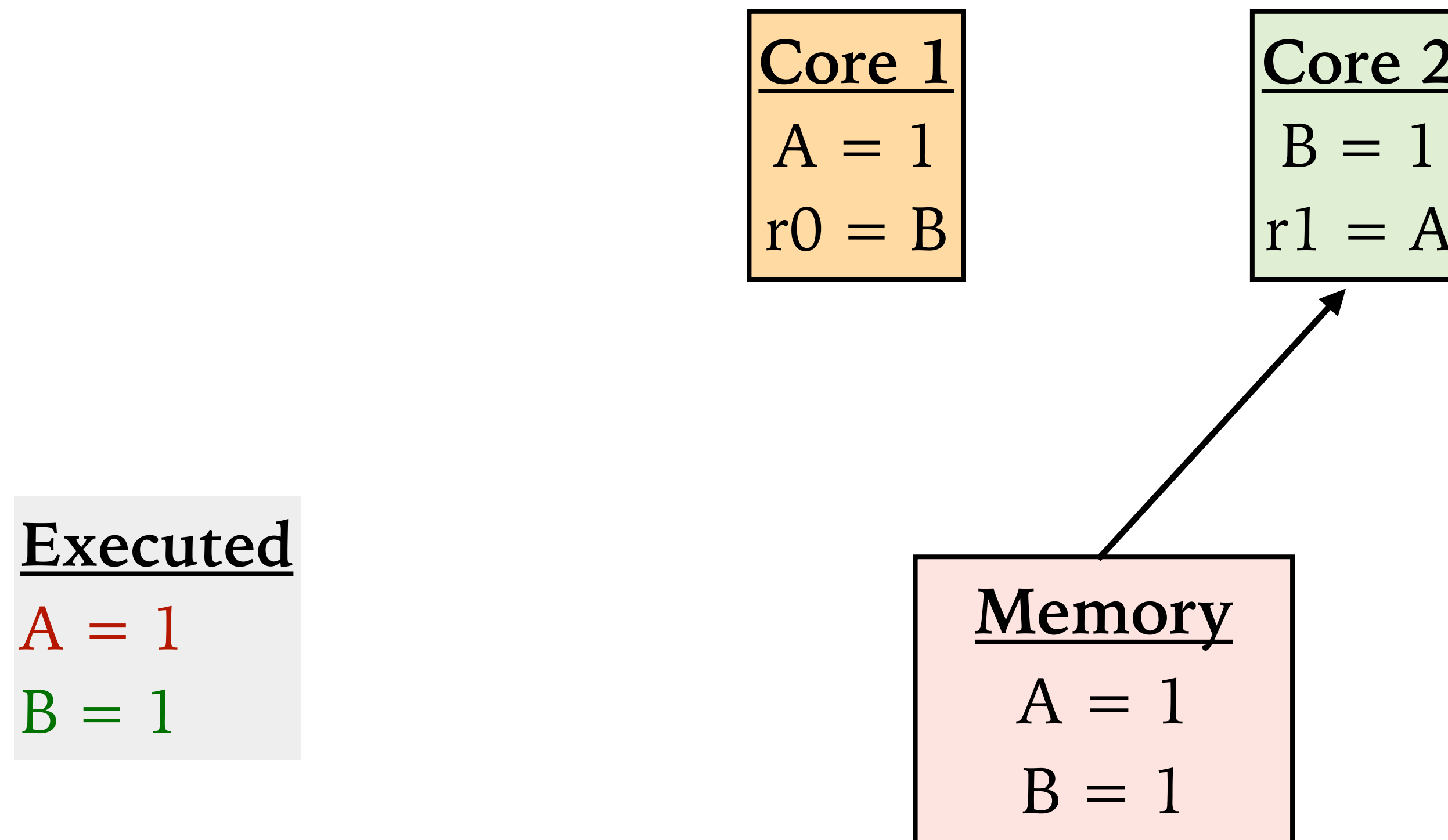
Sequential Consistency

- Can be seen as a “switch” running one instruction at a time from different threads.



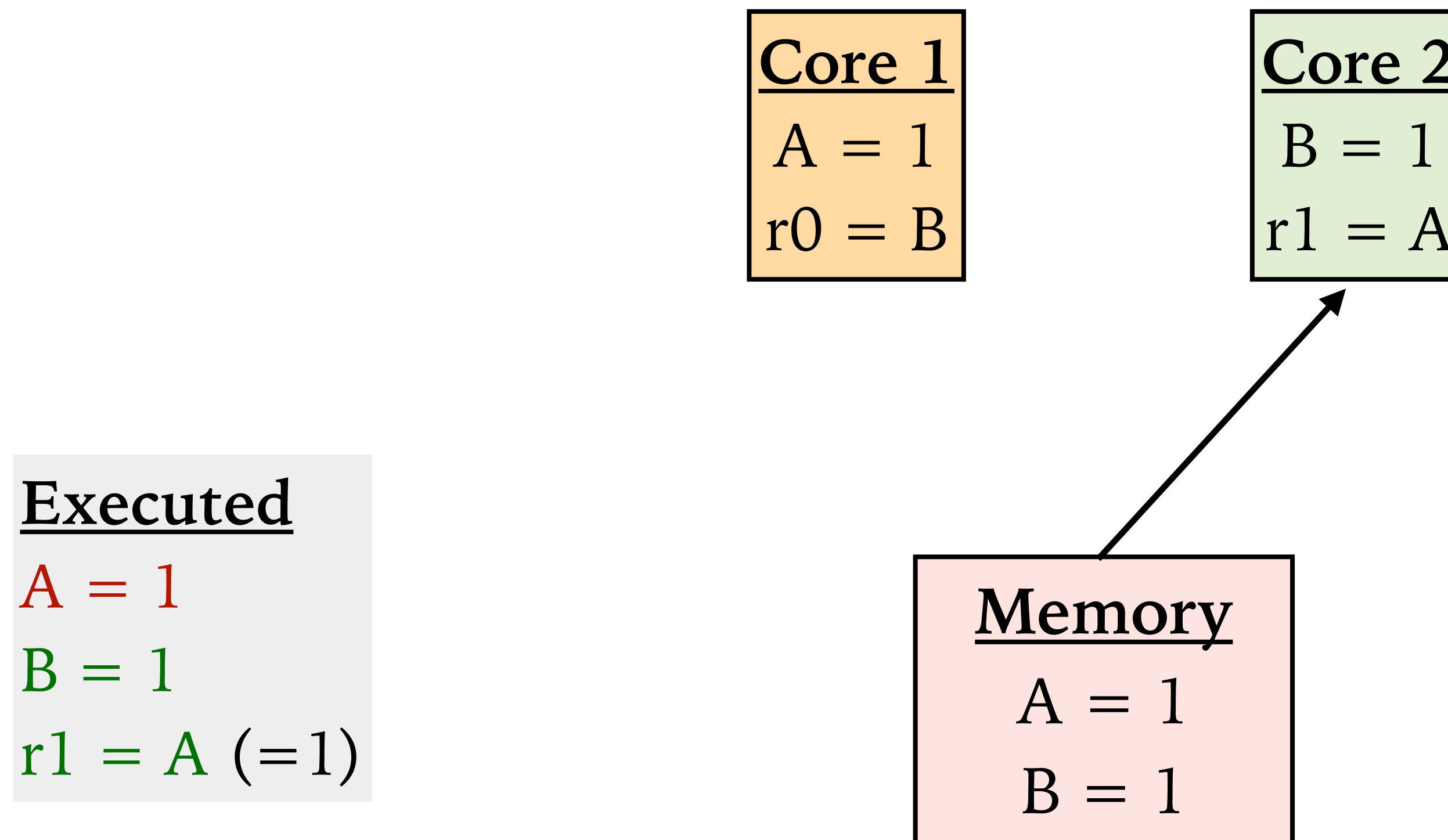
Sequential Consistency

- Can be seen as a “switch” running one instruction at a time from different threads.



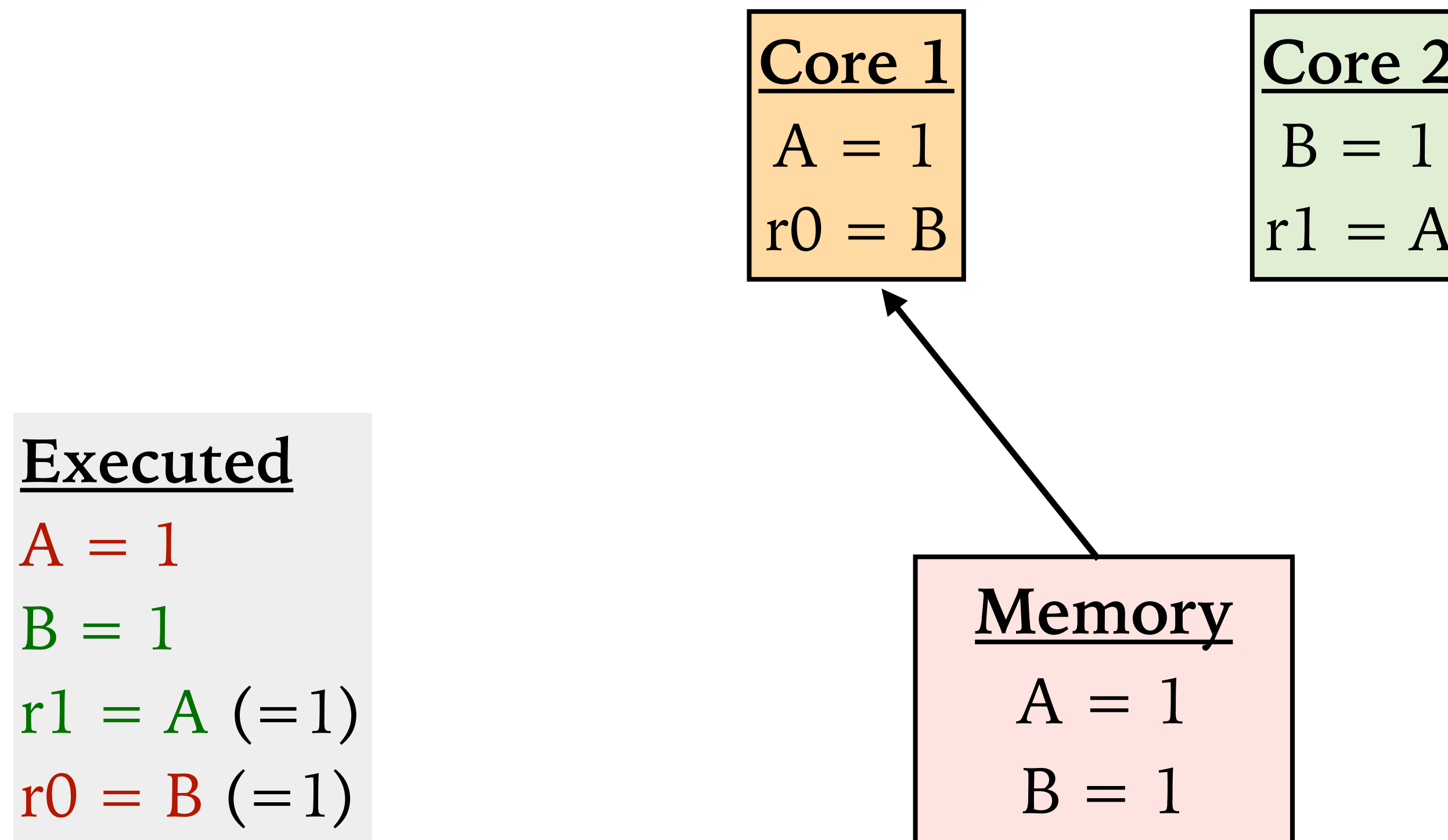
Sequential Consistency

- Can be seen as a “switch” running one instruction at a time from different threads.



Sequential Consistency

- Can be seen as a “switch” running one instruction at a time from different threads.

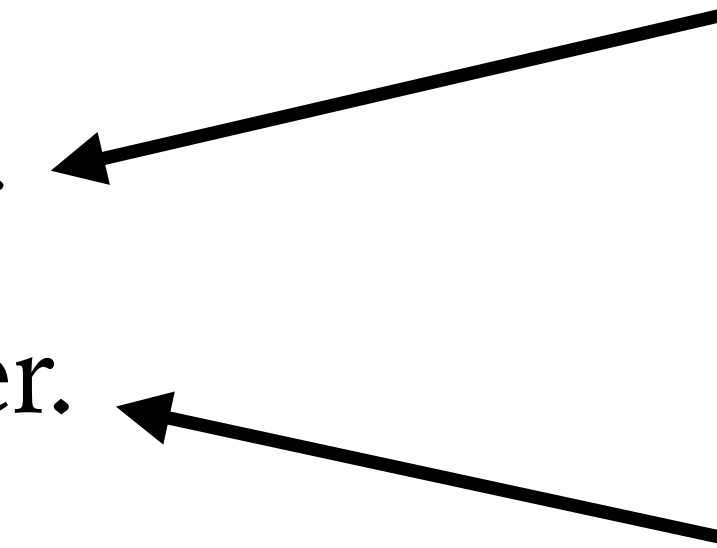


Sequential Consistency

- Two invariants:

- All operations executed in some sequential order.
- Each thread's operations happen in program order.

Order across
threads



Order within
a thread

- Says nothing about **which** order all operations happen in

- Any interleaving of threads is allowed

- Developed by **Leslie Lamport** in 1979 (**Turing Award in 2013**).



Memory (Consistency) Models

- A memory model defines the permitted reorderings of memory operations during execution.
- A **contract between hardware and software**: the hardware will only mess with your memory operations in these ways.
- Sequential consistency is the strongest memory (consistency) model; allows the fewest reorderings.



Why SC?

- Agrees with programmer intuition!

Why not SC?

- Extremely slow to guarantee in hardware!
 - The “switch” model is very conservative and does not allow several valid reorderings.



The Problem with SC

These instructions don't conflict;
we don't really need to
wait for the first to finish.

Executed

$A = 1$

Thread 1

$A = 1$
 $r0 = B$

Thread 2

$B = 1$
 $r1 = A$

Memory

$A = 1$
 $B = 0$

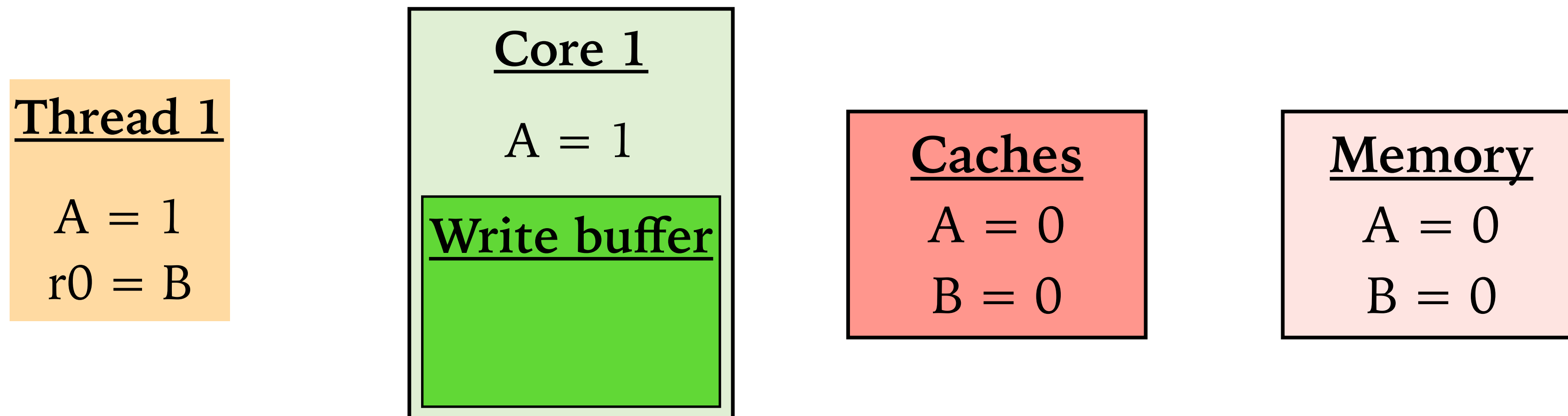
In addition, the whole world of
compiler optimizations depends
on instruction reordering.

Also, writing to memory
takes a lot of time.



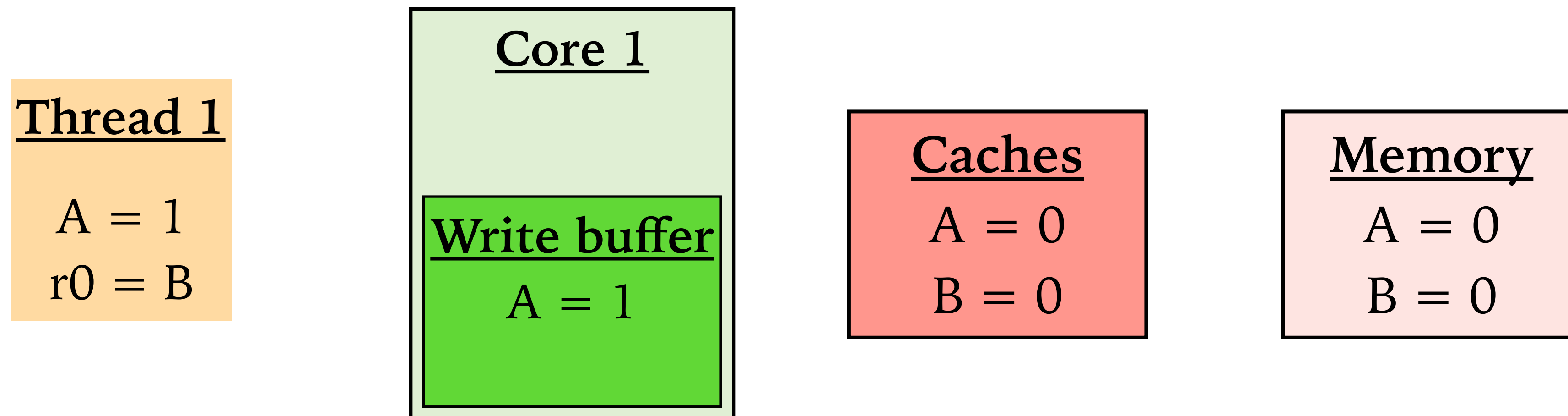
Hardware-Level Optimization: Write Buffers

- Store writes in a processor-local buffer and proceed to next instruction immediately.
- The cache will pull out writes out of the write buffer when it's ready.



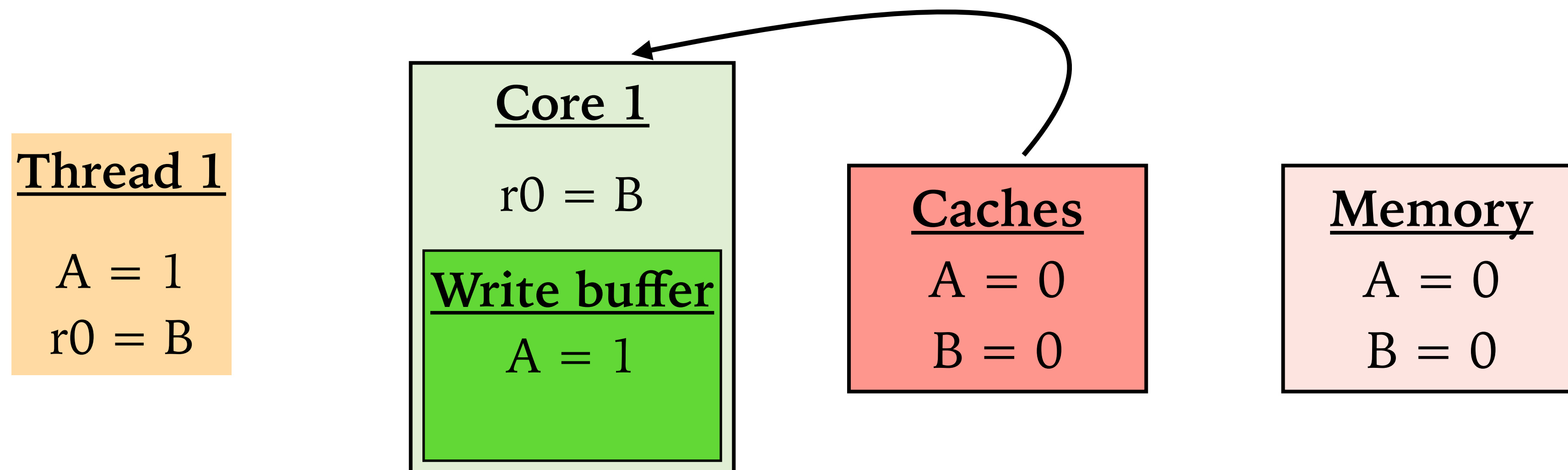
Hardware-Level Optimization: Write Buffers

- Store writes in a processor-local buffer and proceed to next instruction immediately.
- The cache will pull out writes out of the write buffer when it's ready.



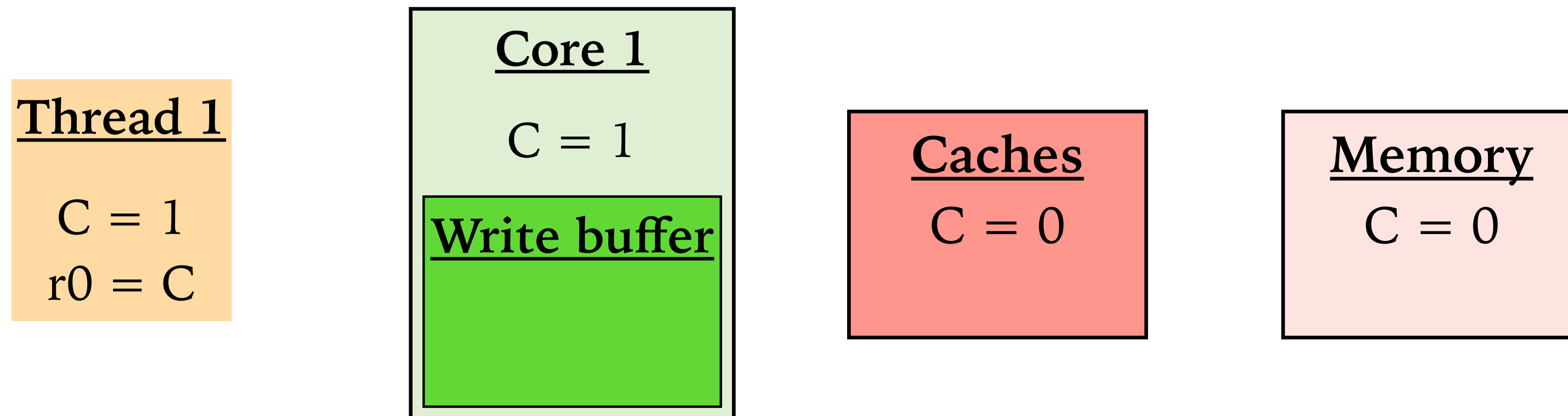
Hardware-Level Optimization: Write Buffers

- Store writes in a processor-local buffer and proceed to next instruction immediately.
- The cache will pull out writes out of the write buffer when it's ready.



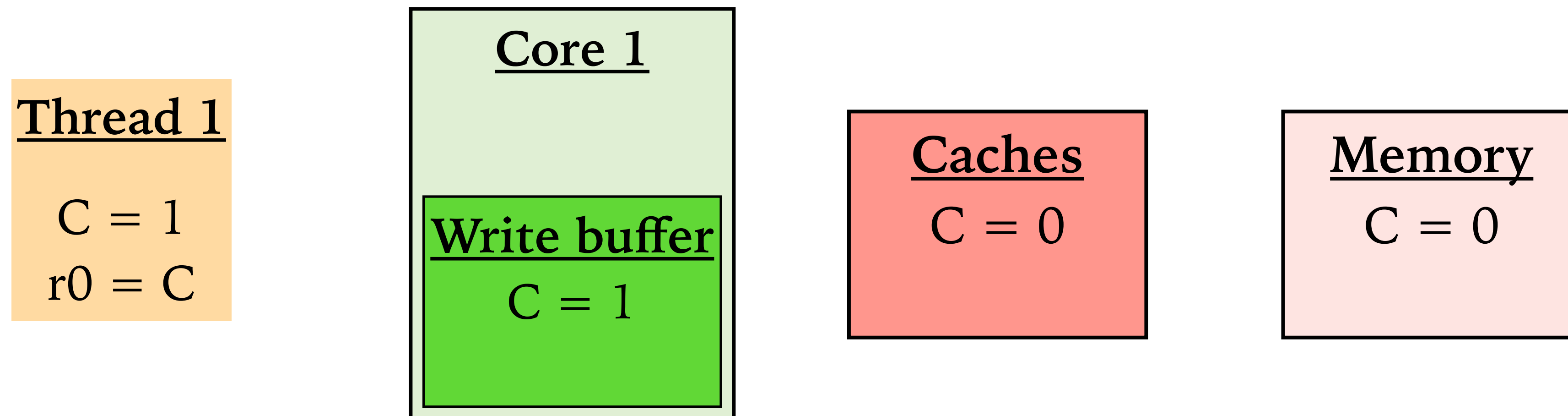
Hardware-Level Optimization: Write Buffers

- Store writes in a processor-local buffer and proceed to next instruction immediately.
- The cache will pull out writes out of the write buffer when it's ready.



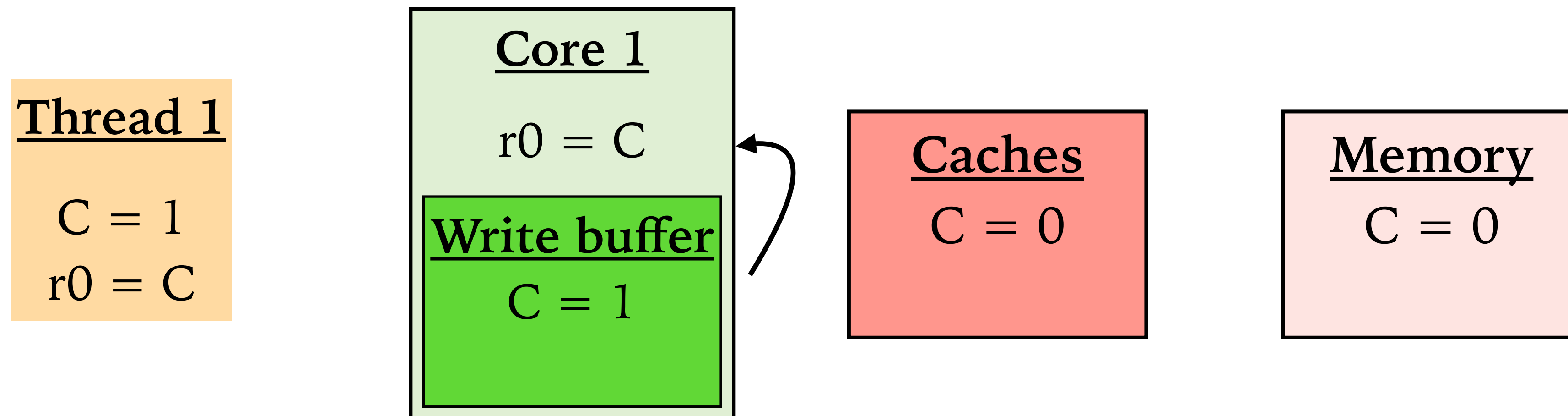
Hardware-Level Optimization: Write Buffers

- Store writes in a processor-local buffer and proceed to next instruction immediately.
- The cache will pull out writes out of the write buffer when it's ready.



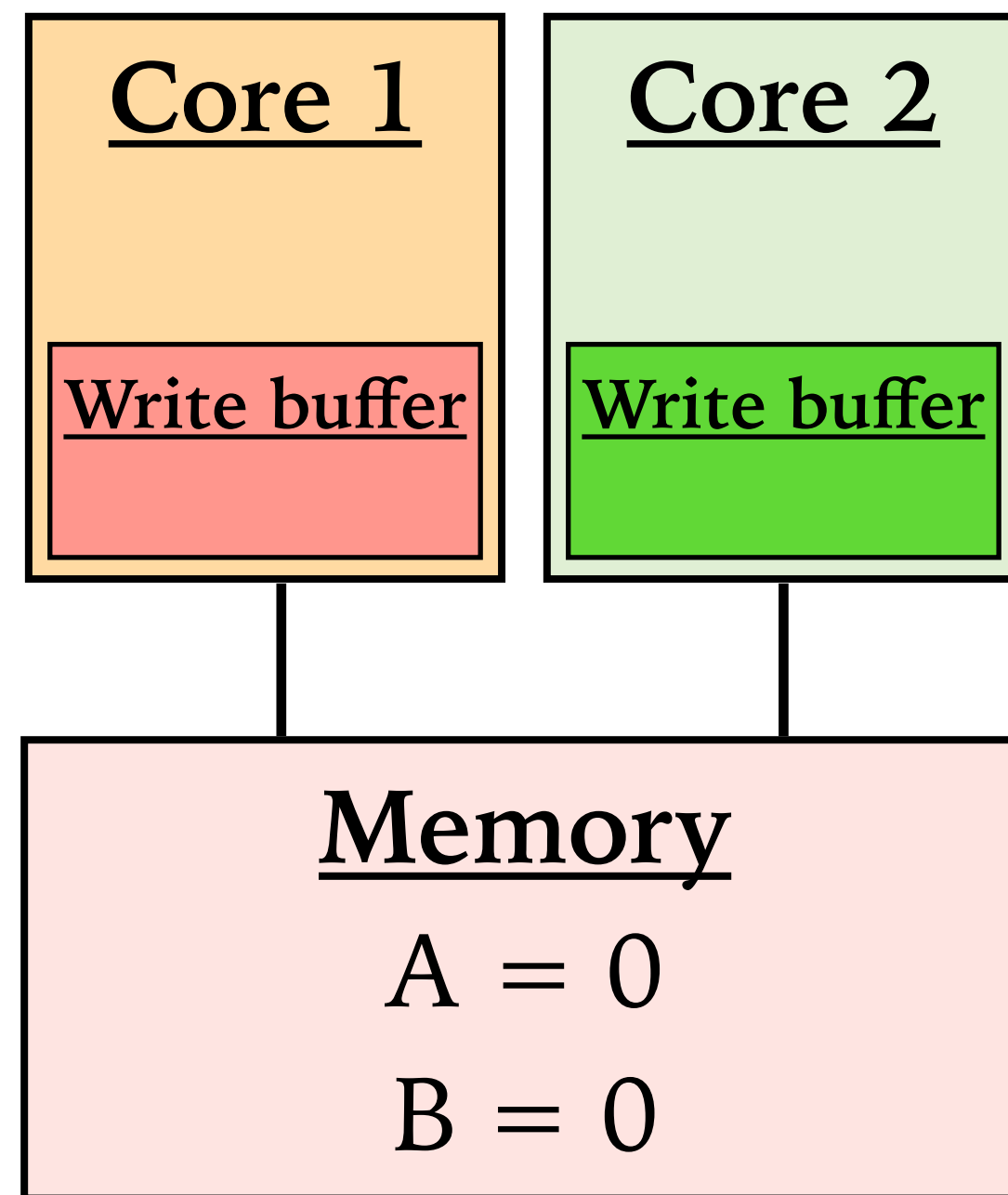
Hardware-Level Optimization: Write Buffers

- Store writes in a processor-local buffer and proceed to next instruction immediately.
- The cache will pull out writes out of the write buffer when it's ready.



- Write buffers define a *new memory model*!

Write buffers change memory behaviour



Thread 1

(1) $A = 1$
(2) $r0 = B$

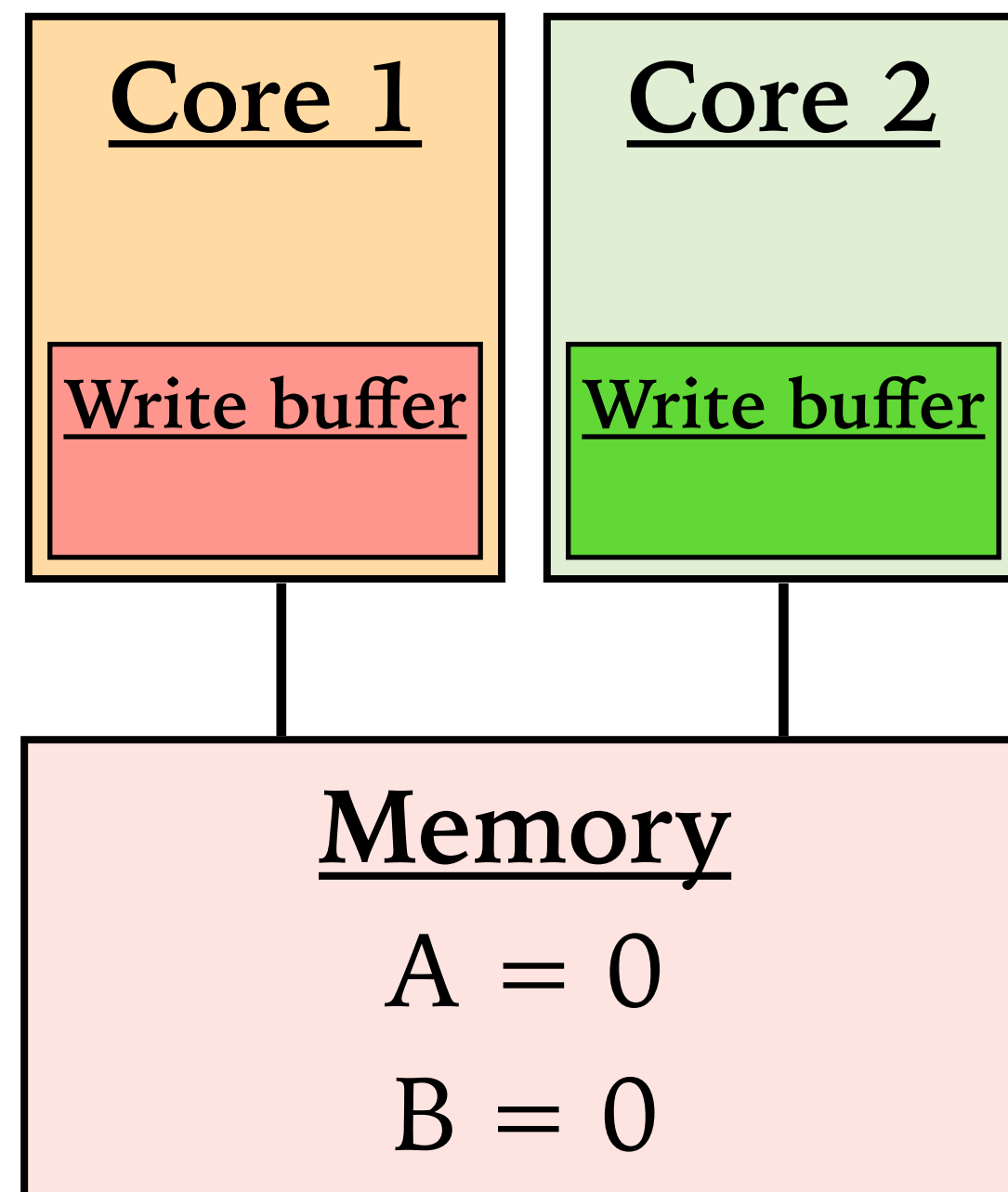
Thread 2

(3) $B = 1$
(4) $r1 = A$

➤ $r0 = 0?$ and $r1 = 0?$

➤ SC? No.

Write buffers change memory behaviour



Thread 1

(1) $A = 1$
(2) $r0 = B$

Thread 2

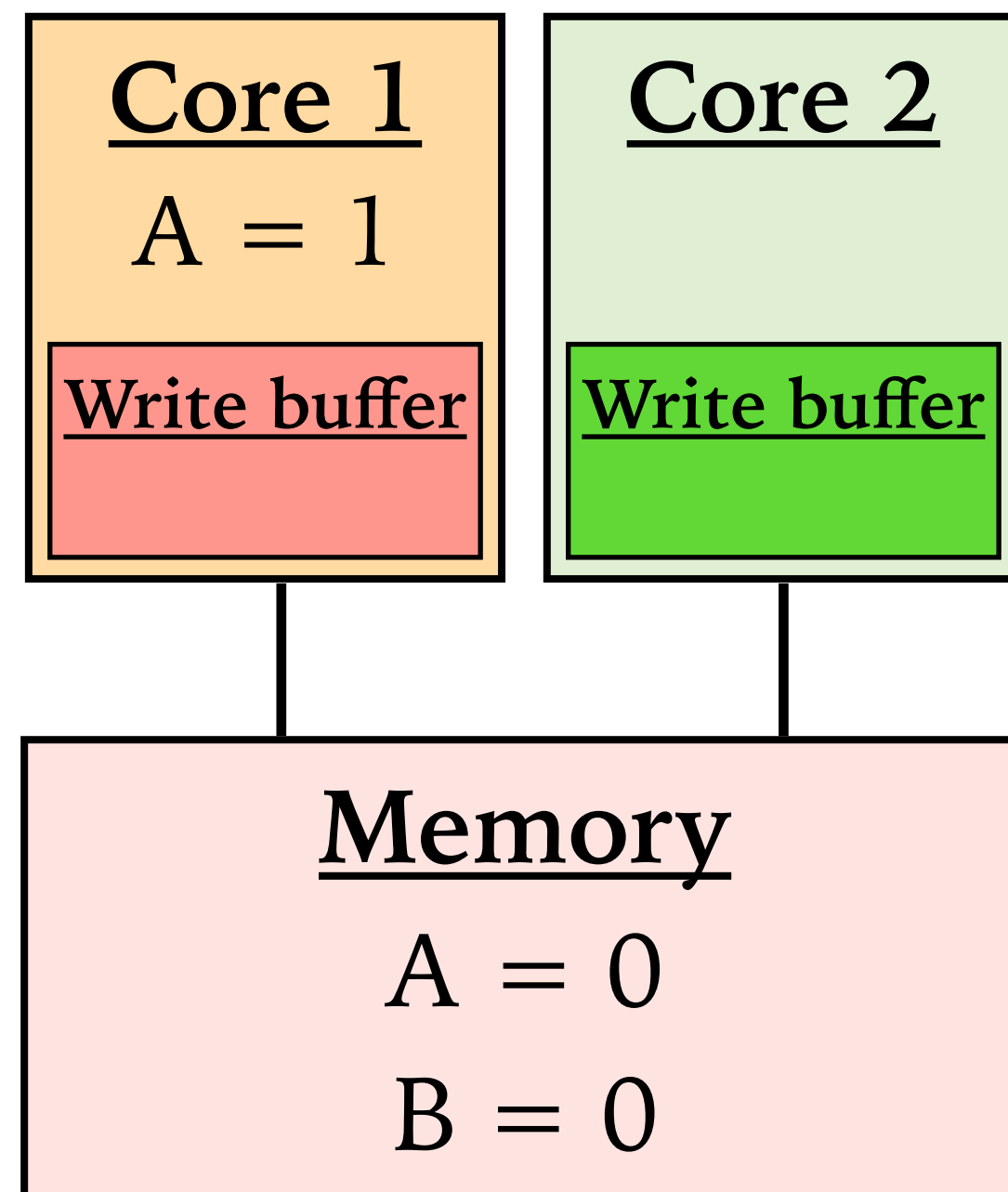
(3) $B = 1$
(4) $r1 = A$

➤ $r0 = 0?$ and $r1 = 0?$

➤ SC? No.

Executed

Write buffers change memory behaviour



Thread 1

(1) $A = 1$
(2) $r0 = B$

Thread 2

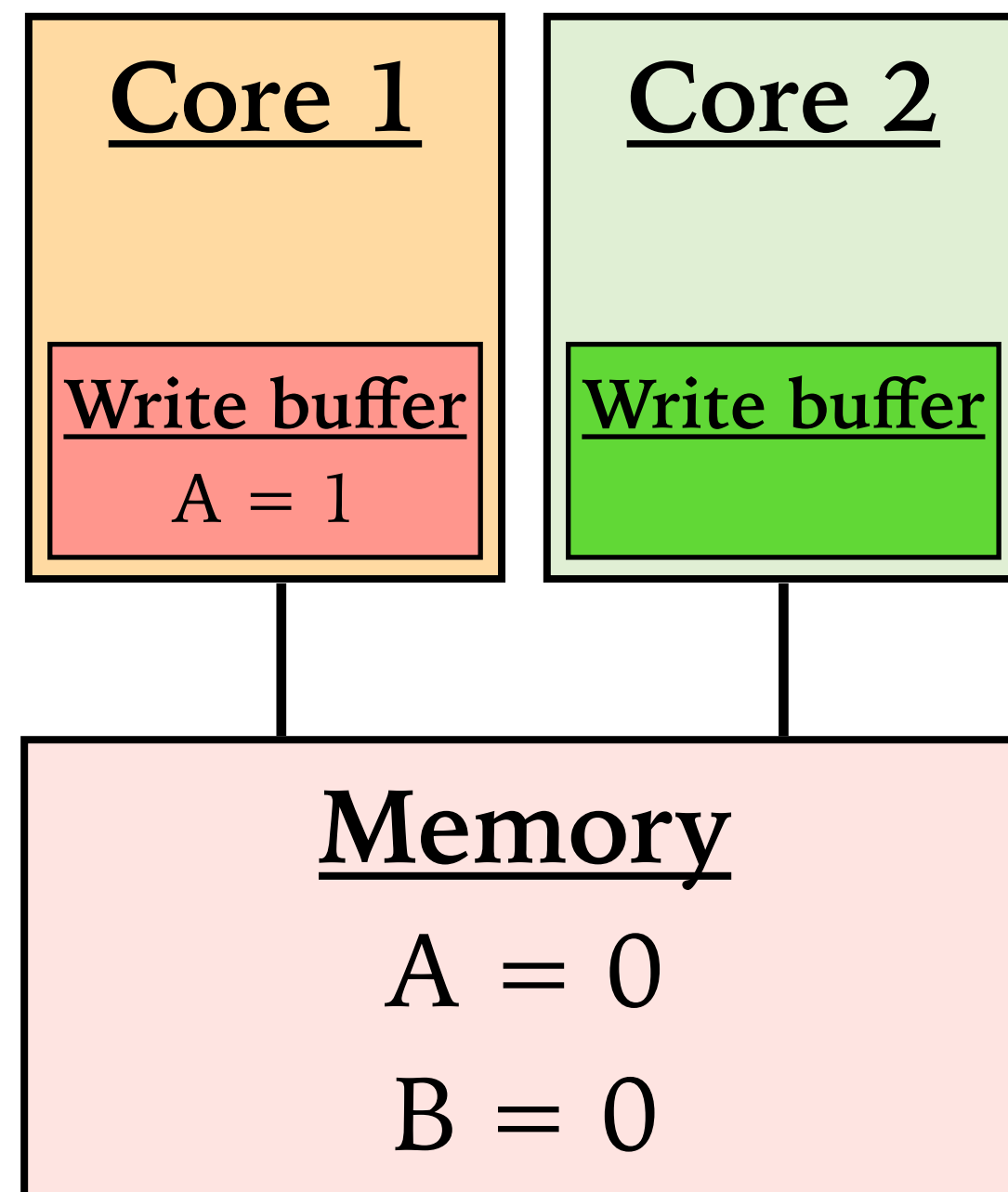
(3) $B = 1$
(4) $r1 = A$

➤ $r0 = 0?$ and $r1 = 0?$

➤ SC? No.

Executed

Write buffers change memory behaviour



Thread 1

(1) $A = 1$
(2) $r0 = B$

Thread 2

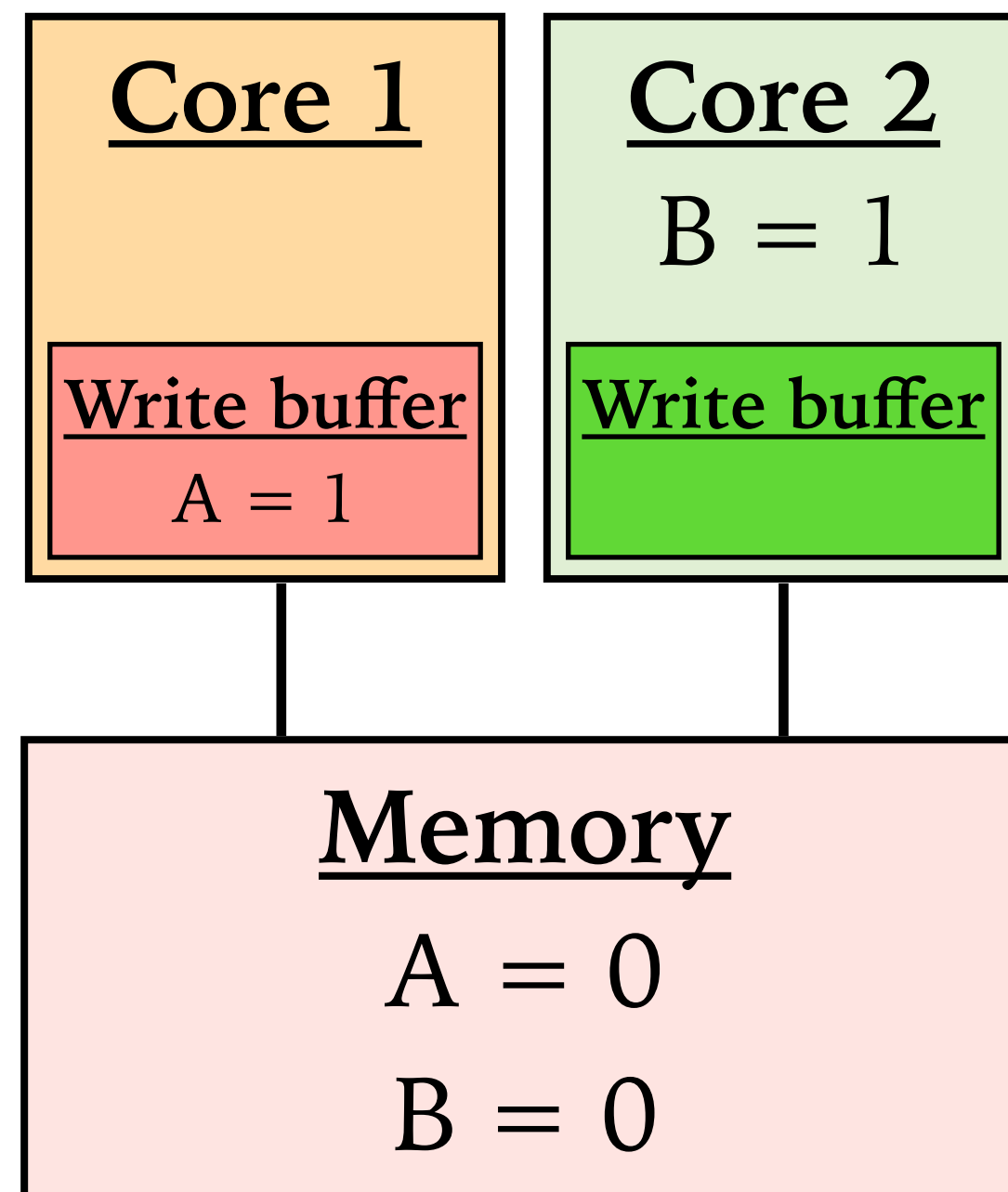
(3) $B = 1$
(4) $r1 = A$

➤ $r0 = 0?$ and $r1 = 0?$

➤ SC? No.

Executed

Write buffers change memory behaviour



Thread 1

(1) $A = 1$
(2) $r0 = B$

Thread 2

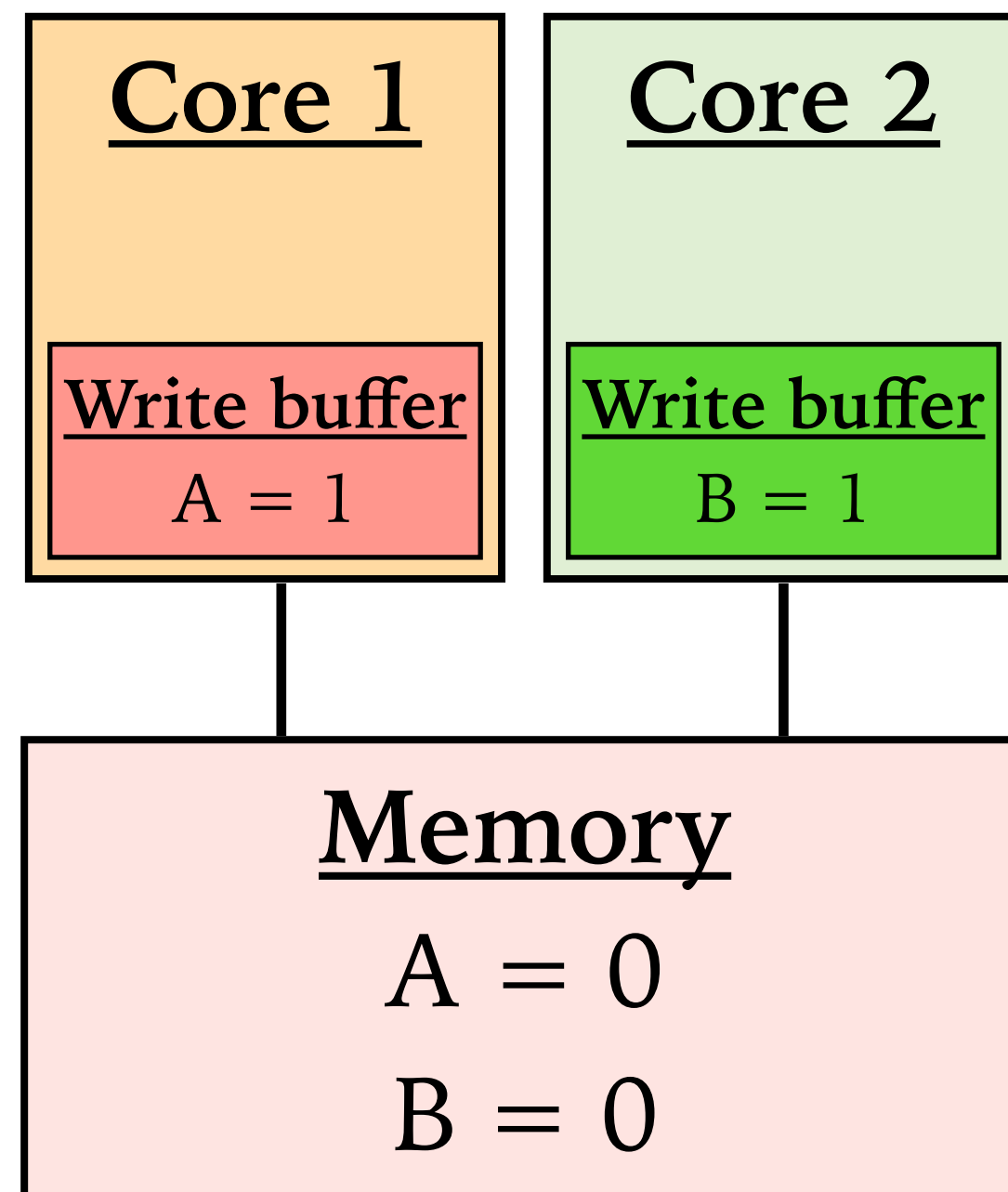
(3) $B = 1$
(4) $r1 = A$

➤ $r0 = 0?$ and $r1 = 0?$

➤ SC? No.

Executed

Write buffers change memory behaviour



Thread 1

(1) $A = 1$
(2) $r0 = B$

Thread 2

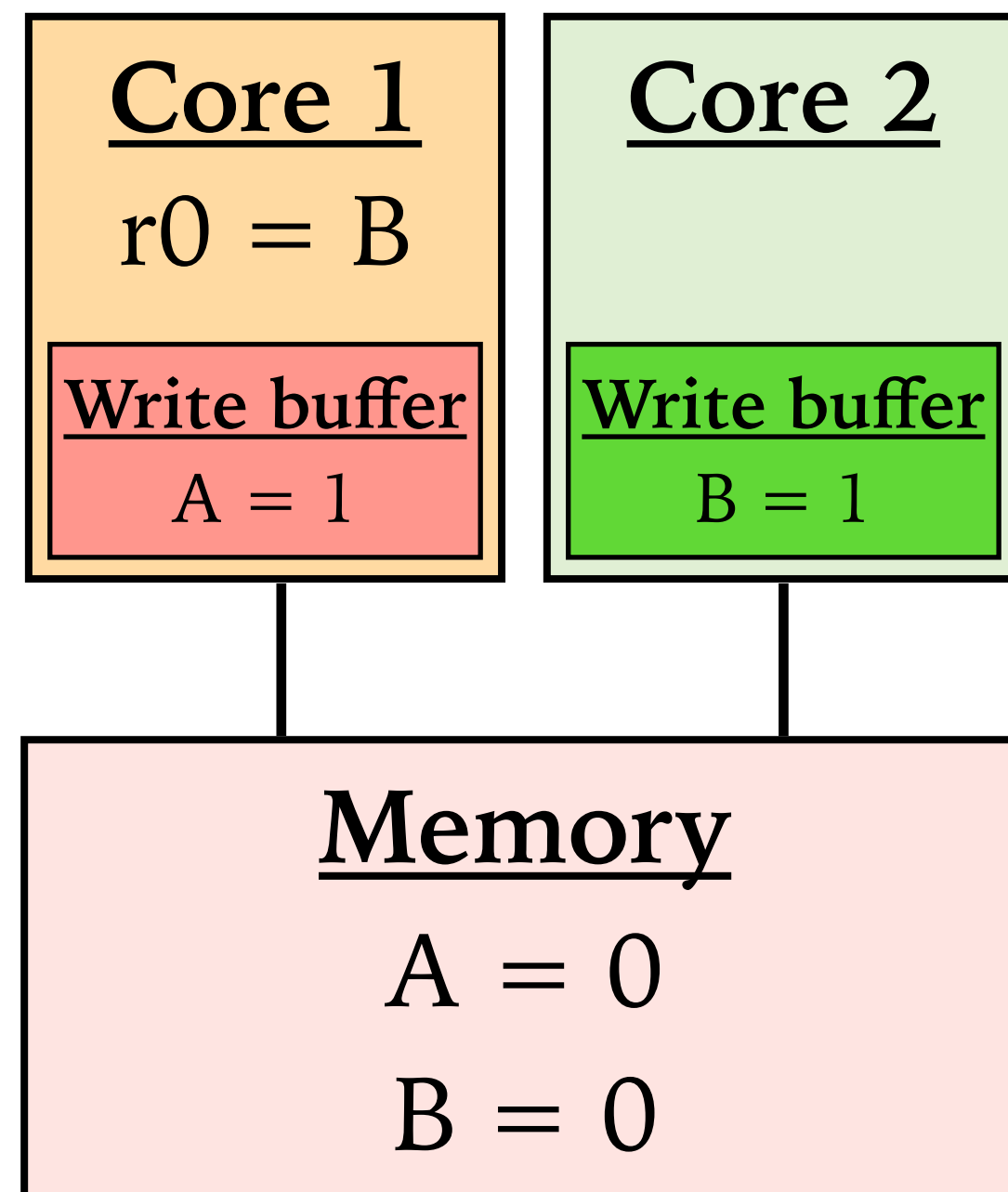
(3) $B = 1$
(4) $r1 = A$

➤ $r0 = 0?$ and $r1 = 0?$

➤ SC? No.

Executed

Write buffers change memory behaviour



Thread 1

(1) $A = 1$
(2) $r0 = B$

Thread 2

(3) $B = 1$
(4) $r1 = A$

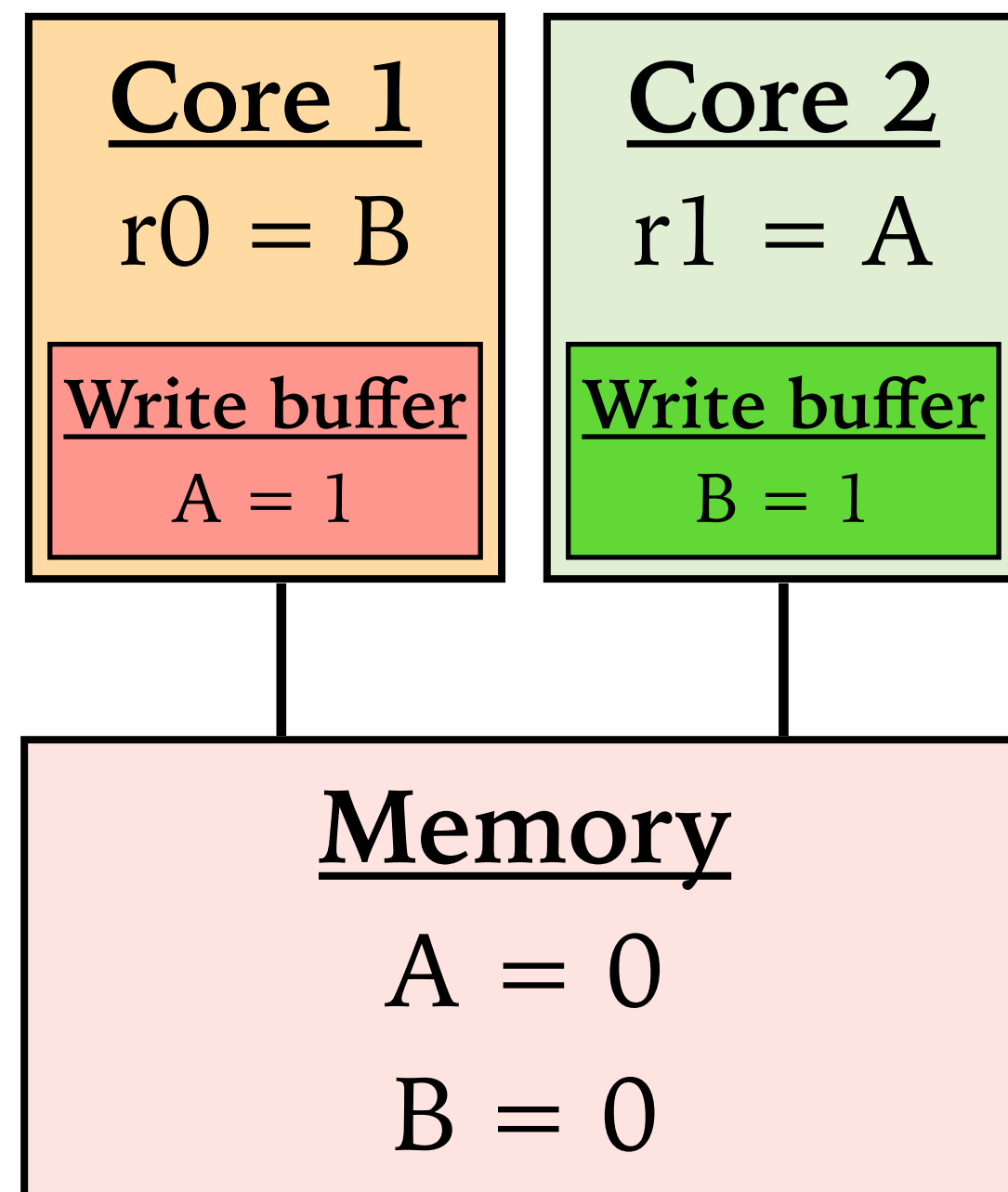
➤ $r0 = 0?$ and $r1 = 0?$

➤ SC? No.

Executed

$r0 = B (=0)$

Write buffers change memory behaviour



Thread 1

(1) $A = 1$
(2) $r0 = B$

Thread 2

(3) $B = 1$
(4) $r1 = A$

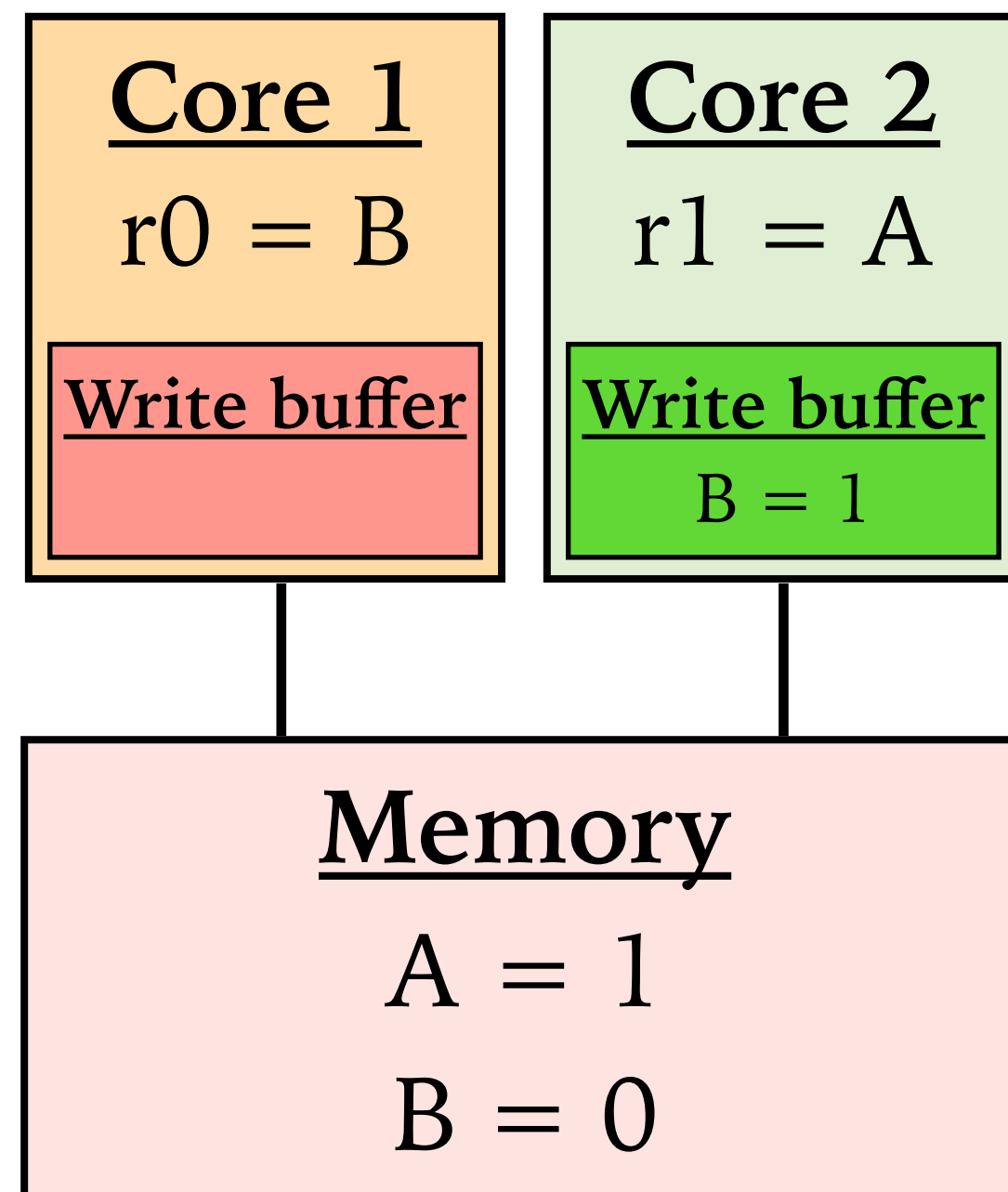
➤ $r0 = 0?$ and $r1 = 0?$

➤ SC? No.

Executed

$r0 = B (=0)$
 $r1 = A (=0)$

Write buffers change memory behaviour



Thread 1

(1) $A = 1$
(2) $r0 = B$

Thread 2

(3) $B = 1$
(4) $r1 = A$

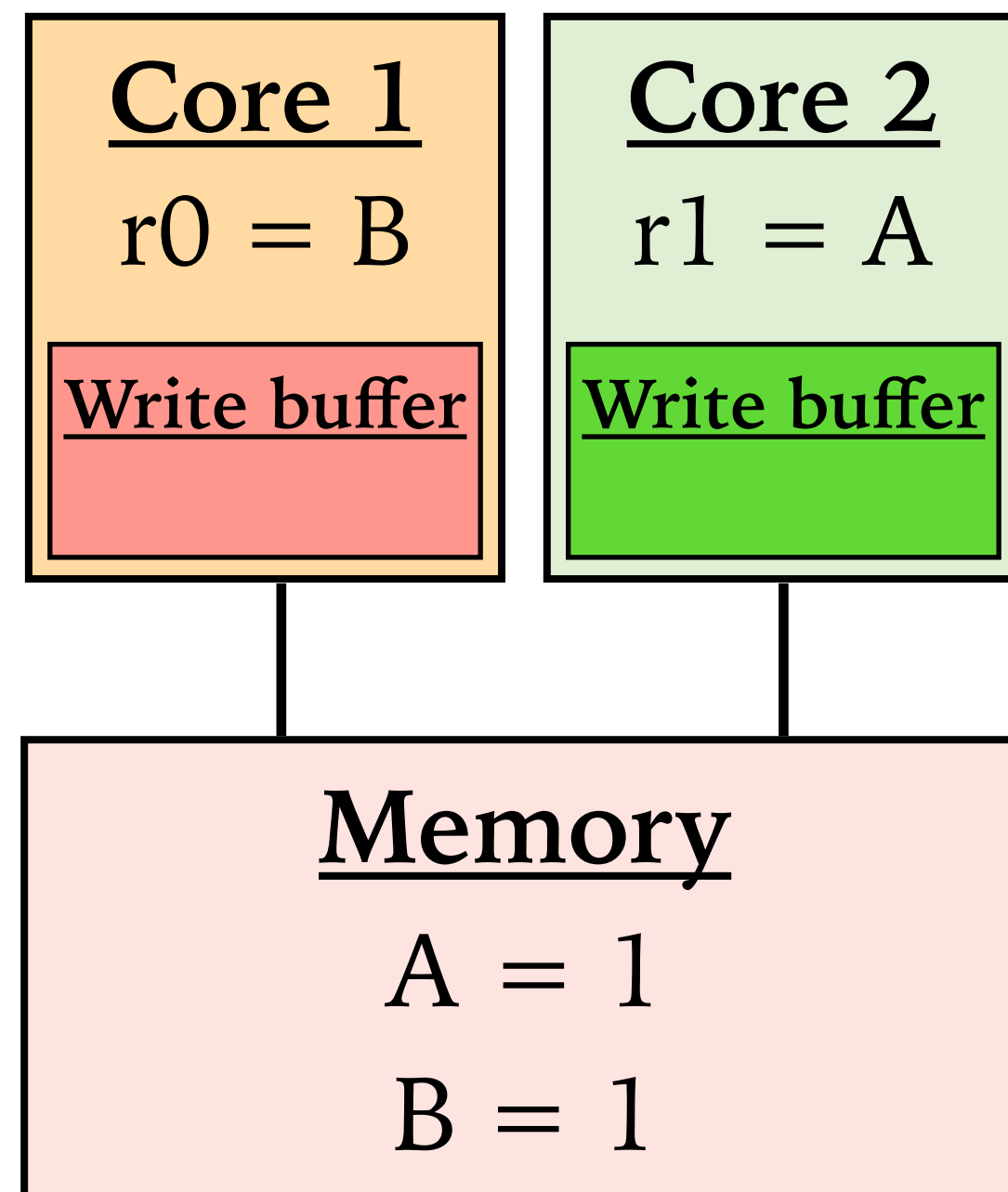
➤ $r0 = 0?$ and $r1 = 0?$

➤ SC? No.

Executed

$r0 = B$ ($=0$)
 $r1 = A$ ($=0$)
 $A = 1$

Write buffers change memory behaviour



Thread 1

(1) $A = 1$
(2) $r0 = B$

Thread 2

(3) $B = 1$
(4) $r1 = A$

➤ $r0 = 0?$ and $r1 = 0?$

➤ SC? No.

➤ Write Buffers? Yes!

Executed

$r0 = B (=0)$
 $r1 = A (=0)$
 $A = 1$
 $B = 1$

Total Store Ordering (TSO)

- TSO allows optimizations that can make the underlying program significantly faster.
- Essentially, SC + Write Buffers.
- But..
 - TSO allows more behaviours than SC.
 - Which means, programs are harder to reason about.
- Yet..
 - x86 specifies TSO as its memory model.



PL Memory Models

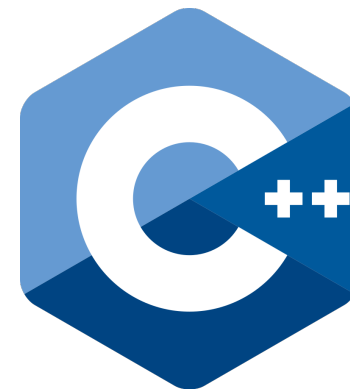
- Guess which was the first programming language to have a memory model?

- Of course:



- And which language followed suit?

- Of course:



- Java and C++ have the most extensive memory-model descriptions till date

- But Java has multithreading in-built, which is good

- Java philosophy: Guarantee sequential consistency for data-race free programs; trust the programmer to insert required synchronization

What all should I do as a programmer?

- Require new ways of thinking
- New paradigm: Concurrency

Two popular ways of writing concurrent programs

➤ Shared-memory concurrency (*imperative roots*)

- Allow variable mutation; control with explicit synchronization
- Supported through atomic instructions, monitors, locks, etc
- Java synchronized blocks, OpenMP critical sections, C++ mutexes

➤ Message-passing concurrency (*functional roots*)

- Shared memory discouraged; communication through passed messages
- Popularly implemented as the Actor model (objects with local state and mailbox)
- Scala's Akka actors, Go's channels (CSP), Erlang's message queues



Hawk's-eye view

Paradigm	Unit of Composition	Time/Order Model	Key Abstraction
Imperative	Statements	Sequential	State and Control
Functional	Functions	Referential Transparency	Values & Expressions
Logic	Facts & Rules	Search Space	Proofs & Constraints
OO	Objects	Message Passing	Encapsulation & Inheritance
Concurrent	Processes & Events	Partial Order / Interleaving	Synchronization & Communication



Problem Solving as a PL Person



How to avoid buffer overflows?

- **HW:** Add extra circuitry; *run-time and space overheads*
- **OS:** Add checks in the ATU; *lot of process-level interruption*
- **ML+FM:** Predict based on past knowledge and then verify; *extremely expensive*
- **PL:**
 - Perform static analysis and identify (un)safe accesses
 - Design a programming language where this is not possible
 - Add a “managed runtime” over the OS-Hardware layer for dynamic analysis
 - *Static analysis is imprecise and dynamic analysis is expensive*
- **CompL:** Analyze what you can statically, and improve the results dynamically



AMA++



AMA++: Marks et al.

“what should be the minimum total marks adding the quizzes midsem and endsem to pass the course (i was not present in the first lecture)”



AMA++: Further learning

“Some of the next interesting courses building upon this course”

“What to do if I absolutely loved the course (with respect to anything extra I can do apart from coursework)?”



AMA++: Experimentation

“Why were Tuesday's slides blue?”

CS339: Abstractions and Paradigms for Programming

Logic Paradigm

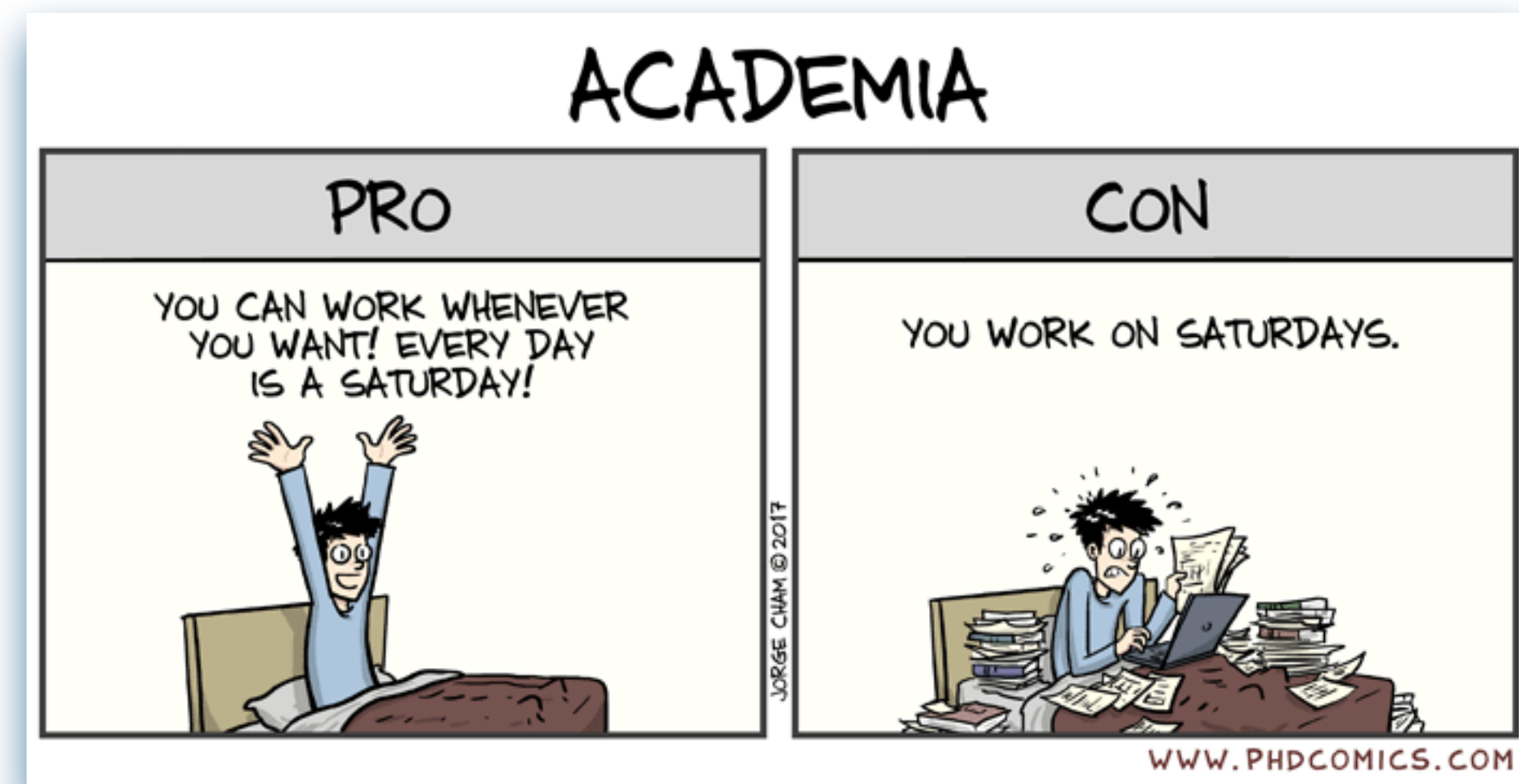
Manas Thakur
CSE, IIT Bombay



Autumn 2025

AMA++: Fun in learning

“Some story behind giving memes as the starting point in exams paper”



The *spirit* of this course (from Lecture 1)

Dedication text of *Structure and Interpretation of Computer Programs (SICP)*

Authors: Harold Abelson and Gerald Jay Sussman with Julie Sussman (MIT)

This book [course] is dedicated, in respect and admiration, to the spirit that lives in the computer:

“I think that it’s extraordinarily important that we in computer science keep fun in computing. When it started out, it was an awful lot of fun. Of course, the paying customers got shafted every now and then, and after a while we began to take their complaints seriously. We began to feel as if we really were responsible for the successful, error-free perfect use of these machines. I don’t think we are. I think we’re responsible for stretching them, setting them off in new directions, and keeping fun in the house. I hope the field of computer science never loses its sense of fun. Above all, I hope we don’t become missionaries. Don’t feel as if you’re Bible salesmen. The world has too many of those already. What you know about computing other people will learn. Don’t feel as if the key to successful computing is only in your hands. What’s in your hands, I think and hope, is intelligence: the ability to see the machine as more than when you were first led up to it, that you can make it more.”

—Alan J. Perlis (April 1, 1922 – February 7, 1990), first Turing Award winner



My spirit as an instructor/advisor/friend

If I could make you feel any of it through this course,
I'd consider myself successful. You surely did. 🍌

...it's extraordinarily important that we in computer science keep fun
in computing...

...we're responsible for stretching them, setting them off in new
directions, and keeping fun in the house...

...the ability to see ... that you can make it more...